# Quinn Gifford

quinngiffordwork@gmail.com • **Website** • LinkedIn • **GitHub**

## EDUCATION

**Oregon State University** — **Corvallis, OR**

**B.S. in Electrical and Computer Engineering, Computer Science Minor | GPA: 3.70** — **June 2027 (Expected)**

## TECHNICAL SKILLS

**Hardware:** Embedded Systems, Firmware, PCB Design, KiCad, Debugging, RTOS, STM32, RP2040, AVR, FPGA, SystemVerilog, I2C, SPI, UART, Verification & Validation, Soldering, Circuit Analysis, Digital Logic Design, Quartus, Simulink, Microchip Studio, LTspice

**Software:** C/C++/C#, Python, Matlab, Java, JavaScript, TypeScript, SQL, GLSL, Linux, React, Node.js, Next.js, CI/CD, Git, Render, Vercel, Stripe, AWS VPS, Pinecone, Claude Code, Claude API, OpenAI API, Cursor

## PROFESSIONAL EXPERIENCE

**Systems Engineer Intern** | **Boeing** — **June 2026 - Present**

- Contributing to verification, validation, and certification of the Earth Reference System and Air Data and Inertial Reference System.
- Used Matlab plotting, Simulink model testing at the logic gate level, and lab simulations to verify system requirements using real flight data and create official documentation to show compliance with FAA regulations.

**Software Engineer Intern** | **Alethian** — **June 2025 - September 2025**

- Deployed a national-scale vector embedding database using Pinecone to store and index pharmacy data enriched with geolocation metadata and entry-type filters, enabling high-performance semantic queries.
- Built a high-speed pharmacy search endpoint combining semantic vector search with geospatial proximity ranking. Was able to achieve **sub-100ms latency** for targeted queries and ~2 second response time for fallback geolocation queries in worst-case scenarios.
- Created performance testing tools for various applications such as insurance card scanning and speech recognition, allowing our team to make up to **18% increases in model accuracy** through iterative improvements.

**Software Engineer (Contract)** | **Scale AI** — **May 2024 - September 2024**

- Completed RLHF tasks, including writing programming related prompts, evaluating model generated code, evaluating model API tool use, and writing high-quality code for training data, in various languages such as Python, Java, JavaScript, Go, PHP, and Verilog.
- Pre-processed GitHub pull requests from SWE-bench subset for LLM coding training using GitHub API and Pandas, resulting in a **12% increase** in resolved programming problems during testing for a model.

**STEM Instructor** | **STEP IT Academy** — **June 2023 - September 2023**

- Instructed classes of up to 10 students on various technology related subjects such as electrical fundamentals, circuit design, microcontrollers, robotics, data structures and algorithms, game development, and Minecraft modding.
- Maintained digital learning resources and managed the organization and storage of hardware components for circuits and robotics.

## PROJECTS (SEE WEBSITE OR GITHUB FOR MORE→) — HTTPS://QUINNGIFFORDWEBSITE.VERCEL.APP/

**Model Rocket Flight Computer**

- Designed and built a STM32F411-based flight data recorder PCB with RTOS firmware, power management, real-time sensor logging, and robust onboard data capture for model rocket flight testing and post-flight analysis.
- Integrated an IMU and TMP117 temperature sensor over I2C and logged flight data to a microSD card over SPI using an RTOS-based task structure for scheduled sampling, buffering, storage, fault handling, and reliable operation under flight conditions.
- Developed firmware for sensor acquisition, timestamping, data buffering, and reliable flash storage, and created a post-flight Python tool to convert logs to CSV, generate plots, and support detailed analysis, debugging, and performance review.
- Performed PCB bring-up, and low-level debugging of I2C, SPI, and GPIO card interfaces using an oscilloscope and circuit debugger.

**Guitar Hero on FPGA**

- Recreated Guitar Hero using a CMOD S7 FPGA to handle game logic and drive a four-strip LED strip display, along with a custom ESP32-based PCB that serves as the wireless game controller.
- Wrote SystemVerilog code on the FPGA to control timing-accurate note generation, input processing, score keeping, and LED-based gameplay visualization, along with C++ firmware on the ESP32 to handle button inputs and bluetooth transmission to the FPGA.
- Created a beatmap generator in Python that converts song audio into an array of sub-millisecond-accurate beat timings, using digital signal processing (DSP) to detect the timing, strength, and instrument classification of each beat. The generator creates multiple difficulty levels and runs in 200 milliseconds. Programmed the FPGA to receive, store, and manage these beatmaps for level selection.